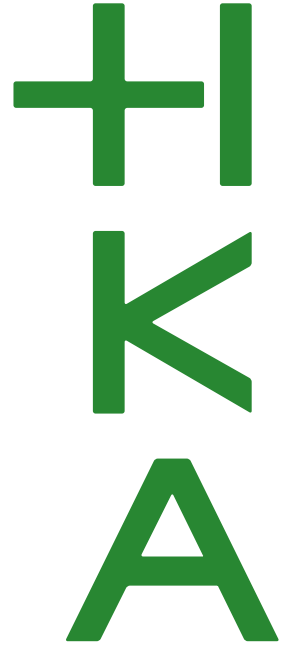


Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Elektro- und
Informationstechnik**

Studiengang Elektro- und Informationstechnik (Bachelor) -
Vertiefung Automatisierungstechnik



Projektarbeit

AR-Datenvisualisierung für Brauanlagen

von
Niklas Felhauer
Jona Wilhelmi
Leon Bunkus

Matrikelnummer
83670
84820
85592

Betreuer: Prof. Dr. Phillip Nenninger
Simon Holzhauer

Vorlesung: Automatisierungstechnik

Inhaltsverzeichnis

1 Zielsetzung	1
2 Grundlagen	2
2.1 Raspberry Pi	2
2.1.1 Access Point	3
2.2 Standard OPC UA	4
2.3 Android App	5
2.3.1 Grundbegriffe	6
3 Datenübertragung	7
3.1 SPS zum Raspberry Pi	7
3.2 Raspberry Pi zum Smartphone	7
3.2.1 MQTT auf dem Raspberry Pi einrichten	7
3.2.2 Testcode auf dem Laptop	7
3.2.3 Acces Point einrichten	8
3.2.4 Passwort für den Datenzugriff	8
4 Android App	9
4.1 MainViewModel	9
4.1.1 LiveData/MutableLiveData	9
4.1.2 MQTT Daten	10
4.1.3 Globale Variablen	10
4.2 Navigation	11
4.3 Die verschiedenen Tabs	12
4.3.1 Kamera-Tab	12
4.3.2 Overview-Tab	13
4.3.3 MQTT-Tab	13
4.3.4 Settings-Tab	14
4.3.5 Promillerechner-Tab	14
4.4 MQTT Verbindungsstatus Overlay	14
5 Objekterkennungsmodell	15
5.1 Vorüberlegungen	15
5.2 Datenset erstellen	15
5.3 Mediapipe Model Maker	17
6 Schluss	20
7 Ausblick	20

Abbildungsverzeichnis

1	Eine vereinfachte Darstellung des Aktivitätslebenszyklus	5
2	Überblick des Datenflusses der App	11
3	Die verschiedenen Tabs der Android App [2]	12
4	Namensgebung der Aktoren/Sensoren	13
6	Farbige Griffhalter aus dem 3D-Drucker	15
7	Bounding Boxen zeichnen mit Roboflow	16
8	Konsolenausgabe während des Trainings	18

1 Zielsetzung

Wie besprochen schicken wir Ihnen unsere Ideen bezüglich unserer Projektarbeit. Wir planen eine App zu entwickeln, welche folgenden Funktionen enthält:

1. Füllstandsanzeige; Sensoren reparieren
2. Zustand der Ventile (momentaner Durchfluss)
3. Temperaturanzeige → farbige Tanks zur Visualisierung
4. augmented reality
5. Promillerechner
6. Kobolde, welche im Topf rühren

2 Grundlagen

2.1 Raspberry Pi

Ein Raspberry Pi ist ein kompakter, kostengünstiger Einplatinencomputer, der ursprünglich von der Raspberry Pi Foundation entwickelt wurde. Dank seiner kompakten Bauweise und seines erschwinglichen Preises ist der Raspberry Pi ein vielseitiges und leistungsfähiges Werkzeug, welches von kleineren Projekten bis hin zu anspruchsvollen industriellen Anwendungen eingesetzt wird.

Hauptmerkmale:

1. **Kompakte Größe:**
Der Raspberry Pi ist etwa so groß wie eine Kreditkarte, was ihn besonders mobil und platzsparend macht.
2. **Vielseitigkeit:**
Er unterstützt eine Vielzahl von Betriebssystemen (z. B. Raspberry Pi OS, Ubuntu) und kann für Aufgaben wie Programmierung, Netzwerkadministration, Steuerung von IoT-Geräten und Multimedia-Anwendungen verwendet werden.
3. **Erschwinglichkeit:**
Mit einem Preis zwischen 12 € (Raspberry Pi Zero) und etwa 100 € (für High-End-Modelle) ist er eine budgetfreundliche Lösung für Projekte.
4. **Community und Support:**
Der Raspberry Pi hat eine große, aktive Community, die zahlreiche Tutorials, Foren und Open-Source-Projekte bereitstellt.

Hardwareeigenschaften:

1. **Prozessor und Arbeitsspeicher:**
 - Der Raspberry Pi verwendet ARM-basierte Prozessoren, die je nach Modell eine unterschiedliche Leistung bieten.
 - Die RAM-Größen variieren von 512 MB bis zu 16 GB, je nach Modell.
2. **Anschlüsse und Schnittstellen:**
 - USB-Ports: Für Maus, Tastatur oder externe Geräte.
 - HDMI-Ausgang: Für die Bildausgabe auf Monitoren oder Fernsehern.
 - Ethernet: Für kabelgebundene Netzwerke
 - Wi-Fi und Bluetooth: Für drahtlose Verbindungen.
 - GPIO-Pins (General Purpose Input/Output): Ermöglichen die Steuerung und Interaktion mit elektronischen Komponenten wie LEDs und Sensoren.
3. **Speicher:**
Anstelle eines internen Speichers verwendet der Raspberry Pi SD-Karten oder MicroSD-Karten für das Betriebssystem und die Daten.
4. **Energieversorgung:**
Es gibt verschiedene Möglichkeiten einen Raspberry Pi mit Strom zu versorgen:
 - USB Power Bank
 - Batterien/Akkus (AA, 9V, Alkaline, Nickel-Metallhydrid-Akkumulator (NiMH))
 - USB Kabel am PC/Laptop

2.1.1 Access Point

Ein Access Point ist ein Gerät, das als Schnittstelle zwischen kabelgebundenen Netzwerken (z. B. einem Router) und drahtlosen Geräten dient. Er ermöglicht es drahtlosen Geräten wie Smartphones, Laptops oder IoT-Geräten, eine Verbindung zum Netzwerk herzustellen.

Hauptfunktionen eines Access Points:

- **Bereitstellung eines drahtlosen Netzwerks (WLAN):**
Der Access Point erstellt ein lokales WLAN, zu dem sich Geräte verbinden können. Das Netzwerk wird durch die SSID (Service Set Identifier, der WLAN-Name) und eine Verschlüsselung gekennzeichnet.
- **Verbindung mit kabelgebundenen Netzwerken:**
Der Access Point ist normalerweise mit einem Router, Switch oder Server über ein Ethernet-Kabel verbunden. So verbindet er die drahtlosen Geräte mit dem bestehenden Netzwerk.
- **Netzwerksicherheit:**
Der Access Point bietet Sicherheitsprotokolle zur Verschlüsselung der Daten.

Es gibt verschiedene Anwendungsbereiche:

- **Erweitern von Netzwerken:**
Ein Access Point wird oftmals eingesetzt, um die Reichweite eines drahtlosen Netzwerks (z.B. in Büros, Gebäuden oder Fabriken) zu erweitern.
- **Erstellung lokaler Netze**
In speziellen Anwendungen, wie bei Industrieanlagen, wird ein lokales WLAN-Netzwerk bereitgestellt, ohne dass eine Verbindung zum Internet notwendig ist. Dies ermöglicht eine direkte Kommunikation zwischen Geräten.
- **Mobile Anwendung:**
Geräte wie Raspberry Pis können als Access Point konfiguriert werden, um ein drahtloses Netzwerk für andere Geräte zu erstellen. Das ist besonders nützlich in Projekten, in denen kein externer Router vorhanden ist.

Funktionsweise eines Access Points:

Ein Access Point arbeitet mit den Standards des IEEE 802.11-Protokolls, welche den Betrieb von WLAN-Netzwerken regeln.

- **Senden und Empfangen von Daten:**
 - Drahtlose Geräte kommunizieren mit dem Access Point über Funk.
 - Der Access Point sendet die empfangenen Daten an das kabelgebundene Netzwerk oder an andere Geräte im WLAN weiter.
- **Netzwerkverbindung:**
 - Der Access Point verwendet Ethernet für die Verbindung mit dem Hauptnetzwerk.
 - Geräte im WLAN erhalten durch den Access Point Zugriff auf Internet oder lokale Ressourcen.

Access Points besitzen verschiedene Vorteile:

- **Drahtlose Verbindung:** Keine Kabel erforderlich, um Geräte zu verbinden.
- **Flexibilität:** Geräte können sich frei im Bereich des WLANs bewegen.
- **Skalierbarkeit:** Mehrere Access Points können kombiniert werden, um größere Netzwerke zu realisieren.
- **Isolierte Netzwerke:** Für spezifische Anwendungen kann ein eigenständiges, lokales Netzwerk eingerichtet werden, das völlig unabhängig vom Internet arbeitet.

2.2 Standard OPC UA

Merkmale von OPC UA:

- **Plattformunabhängigkeit:**
OPC UA ist nicht an ein bestimmtes Betriebssystem oder eine bestimmte Programmiersprache gebunden. Dies ermöglicht die Implementierung auf unterschiedlichsten Plattformen, von kleinen eingebetteten Systemen bis hin zu großen Industrie-PCs.
- **Integration von Daten und Metadaten:**
OPC UA ermöglicht den Austausch von Daten, wie z. B. Sensorwerten oder Maschinendaten, und bietet gleichzeitig Zugriff auf zusätzliche Infos wie Einheiten, Zeitstempel oder Zugriffsrechte.
- **Skalierbarkeit:**
Der Standard ist flexibel und kann sowohl in kleinen Geräten wie einem Sensor als auch in komplexen Produktionsanlagen eingesetzt werden.
- **Sicherheit:**
OPC UA bietet integrierte Sicherheitsmechanismen wie Authentifizierung, Datenverschlüsselung, und Datenintegrität.
- **Interoperabilität:**
OPC UA ermöglicht die Kommunikation zwischen Geräten und Systemen unterschiedlicher Hersteller.
- **Modellierung komplexer Datenstrukturen:**
OPC UA unterstützt die Abbildung komplexer Datenmodelle, sodass Maschinen- und Prozessdaten in einer strukturierten Form bereitgestellt werden können.

Wir haben uns für das MQTT-Protokoll entschieden, welches folgende Vorteile bietet:

1. **Zuverlässigkeit:** MQTT bietet verschiedene Quality of Service (QoS)-Level, die sicherstellen, dass Nachrichten korrekt übertragen werden.
2. **Echtzeitfähigkeit:** Daten werden nahezu in Echtzeit zwischen den Geräten ausgetauscht. Die Echtzeitfähigkeit ist für unser Projekt essenziell.
3. **Flexibilität:** Die Topics können flexibel strukturiert werden, sodass mehrere Datenarten gleichzeitig verarbeitet werden können.
4. **Leichtgewichtig:** Das Protokoll benötigt nur minimale Ressourcen wie Rechenleistung, Speicher und Bandbreite, was es ideal für Geräte wie den Raspberry Pi macht.

2.3 Android App

Die Programmierung der App wird mit dem Programm Android Studio Ladybug in der Version 2024.2.1 Patch 2 verwendet. Als Programmiersprache wird Kotlin gewählt, da diese sich besonders gut für die Entwicklung einer Android App eignet.

In einer Android App wird alles in „Aktivitäten“ organisiert. Dabei kann man sich eine Aktivität wie ein Bildschirm in der App vorstellen. Beispielsweise ist die Startseite eine Aktivität und der Bildschirm, der die MQTT Daten anzeigt eine andere Aktivität. Jede Aktivität ist gleich aufgebaut und hat einen festen Lebenszyklus, der aus folgenden Komponenten besteht. Diese sind auch in Abbildung 1 dargestellt.

1. **onCreate()**: wird aufgerufen, wenn die Aktivität zum ersten Mal erstellt wird
2. **onStart()**: Die Aktivität wird für den Nutzer sichtbar, dieser kann aber noch nicht mit der App interagieren
3. **onResume()**: hier ist die App komplett sichtbar und der Nutzer kann interagieren
4. **onPause()**: eine andere App wird geöffnet und diese Aktivität wird pausiert. Sie ist also nicht mehr im Vordergrund
5. **onStop()**: die Aktivität wird nicht mehr benutzt. Es wird zum Beispiel eine andere Aktivität geöffnet. Jedoch wird die Aktivität nicht zerstört
6. **onRestart()**: wird neu gestartet, wenn der Nutzer zur Aktivität zurückkehrt
7. **onDestroy()**: die Aktivität wird komplett zerstört und vom System entfernt.

Diese Komponenten eignen sich beispielsweise auch sehr gute zum Debuggen. Hier kann überprüft werden in welchem Zustand sich die Aktivität befindet.

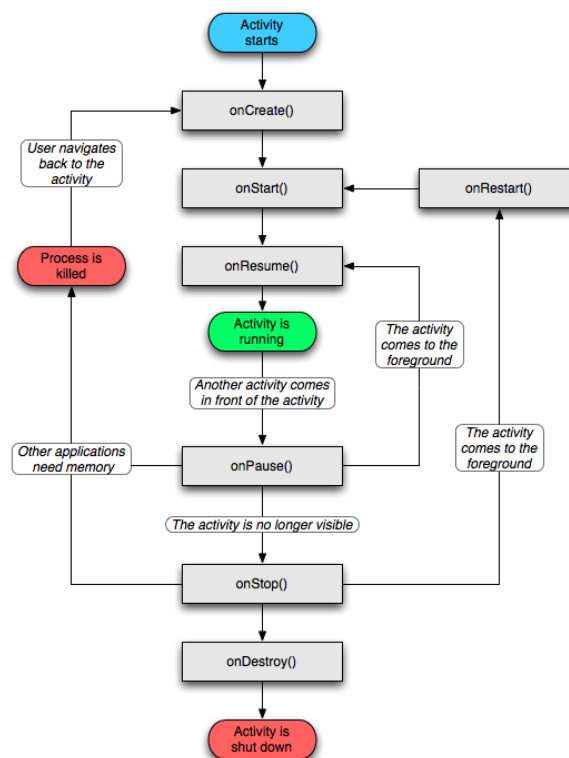


Abbildung 1: Eine vereinfachte Darstellung des Aktivitätslebenszyklus

2.3.1 Grundbegriffe

Activity Eine Activity ist das „Fenster“ der Android-App. In dieser Projektarbeit wird die Single-Activity-Architektur gewählt. Es gibt demnach nur eine Haupt-Activity, die den gesamten Bereich bereitstellt und übergreifende Elemente wie das MQTT-Verbindungs-Overlay enthält.

Fragment Ein Fragment ist ein eigenständiger Bildschirmbereich innerhalb der Activity. Jeder Tab (Camera, Overview, MQTT, Settings und Promillerechner) ist beispielsweise ein Fragment. Diese lassen sich austauschen bzw. zwischen den Tabs kann gewechselt werden, ohne dass die Activity im Hintergrund neu gestartet werden muss.

ViewModel Das ViewModel ist die „Daten- und Logikzentrale“ der App, die unabhängig vom UI-Lebenszyklus arbeitet. Es enthält Daten, die bei einem Tab-Wechsel bestehen bleiben sollen und nicht bei einem Tab-Wechsel verloren gehen.

LiveData Hierbei handelt es sich um eine Klasse für beobachtbare Datenhalter. Wenn sich Werte im ViewModel ändern (z.B. neue MQTT-Daten), dann werden alle UI-Elemente automatisch informiert und aktualisiert, welche diese LiveData abonniert haben. Dadurch werden die Darstellungen immer dann aktualisiert, wenn neue Daten vorliegen.

NavHost und NavController Der NavHost ist ein Container, der immer genau eine Fragment anzeigt. Dabei steuert der NavController, welches Fragment im NavHost sichtbar ist. Die Steuerung in der App erfolgt über die Tab-Leiste oben. Ein Tipp auf ein Symbol teilt dem NavController mit, welches Fragment geladen werden soll.

CameraX Die CameraX Bibliothek liefert den Live-Kamerastream und analysierbare Einzelbilder.

3 Datenübertragung

Die Datenübertragung erfolgt über zwei Wege, zunächst müssen die gemessenen Daten von der SPS zum Raspberry Pi gelangen. Anschließend wird der Raspberry Pi über ein LAN-Kabel mit einem WLAN-Router verbunden. Der Router spannt ein lokales Netzwerk auf, über welches die Daten zum Smartphone weitergeleitet werden, insofern das Smartphone mit dem Router verbunden ist.

3.1 SPS zum Raspberry Pi

Die gemessenen Daten werden über ein Ethernet-Kabel mit dem Protokoll OPC UA von der SPS auf den Raspberry Pi übertragen. Diese werden danach mit einem Python Skript empfangen und aufbereitet.

3.2 Raspberry Pi zum Smartphone

Die Datenübertragung vom Raspberry Pi zum Smartphone läuft über einen Router, der als Access Point fungiert. Dabei wird das Protokoll MQTT (Message Queuing Telemetry Transport) genutzt.

3.2.1 MQTT auf dem Raspberry Pi einrichten

Zunächst muss der Mosquitto MQTT Broker auf dem Raspberry Pi in der Konsole installiert werden:

```
1 sudo apt install -y mosquitto mosquitto-clients
```

Mosquitto als Dienst starten und aktivieren:

```
1 sudo systemctl enable mosquitto
2 sudo systemctl start mosquitto
```

Der aktuelle Status des Mosquitto-Service, kann mit folgendem Code überprüft werden:

```
1 sudo systemctl status mosquitto
```

Zur Übertragung der Daten vom Raspberry Pi zum Smartphone wird eine Python-Skript erstellt, das im GitHub-Repository im Ordner 03.Raspberry_Pi verfügbar ist. Es wird die Verbindung zur SPS mit OPC UA aufgebaut, die Daten aufbereitet, eine MQTT Verbindung hergestellt und die Daten per MQTT auf dem Topic brauanlage/data gepublished.

Anschließend kann auf dem Raspberry Pi überprüft, ob die MQTT Daten lokal auf dem Pi empfangen werden können. Dazu muss in der Konsole dem Topic gefolgt werden mit:

```
1 mosquitto_sub -h localhost -t brauanlage/data
```

Anschließend werden die gesendeten Daten aufgelistet.

3.2.2 Testcode auf dem Laptop

Nun kann weiter getestet werden, ob die MQTT Verbindung auch auf einem Gerät im selben Netzwerk funktioniert, zum Beispiel einem Windows 11 PC. Zuerst muss in der Konsole die MQTT Bibliothek installiert werden mit:

```
1 pip install paho-mqtt
```

Nun kann der MQTT-Dienst gestartet werden, indem im Installationsverzeichnis die Konfigurationsdatei ausgeführt wird. Standardmäßig befindet sich dieses Verzeichnis unter C:/Program Files/mosquitto.

```
1 mosquitto -c mosquitto.conf -v
```

Falls erforderlich, sollte zusätzlich der Port 1883 in der Windows-Firewall freigegeben werden. Anschließend lässt sich auf dem Laptop (Windows 11, 64-Bit) das Testprogramm auf dem GitHub ausführen. Es gibt ein Testprogramm zu senden und ein weiteres zum Empfangen im Ordner 01_MQTT_PC

1. Empfangen: mqtt_sub.py

2. Senden: mqtt_pub.py

3.2.3 Acces Point einrichten

Zur Einrichtung des Acces Points ist der Router zunächst auf Werkseinstellung zurückgesetzt und es wird ein WLAN-Name und Passwort festgelegt. Anschließend wird überprüft, ob der DHCP Server aktiviert ist. Dies ermöglicht es Geräten, automatisch eine IP-Adresse vom Router zu erhalten.

Der Raspberry Pi wird über ein Ethernet-Kabel direkt mit dem Router verbunden und erhält eine statische IP-Adresse. Diese Einstellung wird in den Netzwerkeinstellungen des Router vorgenommen, die allgemein unter `http://192.168.178.1` bzw. in unserem Fall unter `http://fritz.box` zu finden ist.

3.2.4 Passwort für den Datenzugriff

Für die Sicherstellung, dass nicht jeder Zugriff auf die Daten der SPS erhält, wird ein Passwort erstellt.

Erstelle ein Passwort für den Benutzer braumeister:

```
1 sudo mosquitto_passwd -c /etc/mosquitto/passwordfile braumeister
```

Man wird aufgefordert ein Passwort zu vergeben

Passwort: Lga234Vm-7fg

Starte Mosquitto neu:

```
1 sudo systemctl restart mosquitto
```

Im Python-Client kann man sich wie folgt authentifizieren:

```
1 client.username_pw_set("braumeister", "Lga234Vm-7fg")
```

4 Android App

Die App verbindet die Brauanlage im Automatisierungstechnik-Labor mit dem Smartphone. Sie zeigt Live-Messdaten (z.B. Tanktemperaturen, Ventil-/Pumpenzustände), die per MQTT von einer SPS empfangen werden. Außerdem erkennt sie die drei großen Brauanlagen-Tanks über die Kamera (Objekterkennung) und bietet individuelle Einstellmöglichkeiten. Technisch nutzt die App eine Single-Activity-Architektur, das bedeutet, es gibt eine einzige Hauptseite (MainActivity), die mehrere Bildschirme (Fragmente) beinhaltet, zwischen denen per Tab-Leiste gewechselt werden kann. Ein MainViewModel hält Daten und Einstellungen zentral, damit die Daten beim Tab-Wechsel erhalten bleiben. Änderungen werden über LiveData automatisch in allen Tabs sichtbar und stetig aktualisiert.

4.1 MainViewModel

Das MainViewModel bildet die zentrale Datenschnittstelle der App. Es handelt sich um eine Android-Komponente, die Daten unabhängig vom User Interface (UI) im Hintergrund verwaltet. Das ist notwendig, weil die einzelnen Tabs beim Wechseln neu geladen werden und lokale Variablen verloren gehen würden. Das ViewModel speichert die Werte zentral und stellt sie allen Tabs zur Verfügung. Diese Variablen liegen grundsätzlich in zwei Datentypen vor:

1. LiveData/MutableLiveData
2. Globale Variablen

4.1.1 LiveData/MutableLiveData

Dieser Datentyp wird für dynamische Werte verwendet, die sich häufig ändern (z.B. MQTT Nachrichten). Sie sind privat im MainViewModel beschreibbar und global (in den Tabs) nur lesbar. Dadurch wird eine eindeutige Richtung des Datenflusses festgelegt, nämlich:

MainViewModel → Tabs

Zusätzlich unterscheidet man noch zwischen LiveData und MutableLiveData, es werden im MainViewModel immer beide Varianten deklariert, wie im folgenden Codeausschnitt für die Variable `mqttMessages` ersichtlich ist:

```
1 // MutableLiveData:
2 private val _mqttMessages = MutableLiveData<List<String>>(emptyList())
3
4 // LiveData:
5 val mqttMessages: LiveData<List<String>> get() = _mqttMessages
```

Im Code wird dieser Unterschied konsistent mit einem Unterstrich vor dem Variablennamen gekennzeichnet. LiveData haben den normalen Namen (z.B. `mqttMessages`) während MutableLiveData einen Unterstrich vor dem Namen haben (z.B. `_mqttMessages`)

Im MainViewModel liegt zu jeder LiveData-Variable auch immer eine MutableLiveData-Variable vor. Diese MutableLiveData-Variable ist ausschließlich nur im MainViewModel verfügbar und kann auch nur dort beschrieben werden. Für die Tabs gibt es öffentliche Variablen vom Typ LiveData. In den Fragments (Tabs) können diese LiveData mit `observe(...)` beobachtet werden. Bei einer Änderung wird der Wert im Tab automatisch geändert und es muss keine manuelle Aktualisierungs-Funktion implementiert werden. Im folgenden Codeausschnitt registriert sich das Fragment (Tab) mit `observe(viewLifecycleOwner)` als Beobachter und die UI (hier: `textView` bzw. eine Textansicht) zeigt sofort die aktuellen Werte an, ohne dass ein manuelles Neuladen notwendig ist.

```
1 viewModel.mqttMessages.observe(viewLifecycleOwner) { messages ->
2     textView.text = messages.joinToString("\n")
3 }
```


4.1.2 MQTT Daten

Eingehende MQTT Nachrichten werden im JSON-Format empfangen, geparkt und anschließend in verschiedene LiveData-Variablen aufgeteilt:

```

1 override fun messageArrived(topic: String?, message: MqttMessage?) {
2     val raw = message?.toString() ?: return
3     val obj = JSONObject(raw)
4     val parsed = mutableMapOf<String, Float>()
5     obj.keys().forEach { key ->
6         parsed[key] = obj.getDouble(key).toFloat()
7     }
8     _allValues.postValue(parsed)
9 }

```

Dabei entstehen die folgenden drei Datensammlungen:

- **_mqttMessages:** Log aller empfangenen Nachrichten
- **_allValues:** alle Werte aus der MQTT Nachricht zu Tanktemperaturen, Ventile, Pumpen,...
- **_tankTemperatures:** nur die Temperaturen der Tanks

Hierbei sind alle drei als MutableLiveData im MainViewModel verwaltet und sind für die Tabs über den observe(...)-Befehl als LiveData zugänglich.

4.1.3 Globale Variablen

Neben den dynamischen Sensordaten über LiveData verwaltet das MainViewModel auch die Einstellungen für die Objekterkennung, die im Settings-Tab angepasst werden können. Dazu gehören die Auswahl des Modells, die Threshold, die Anzahl der maximal erkennbaren Objekte sowie die Wahl des Delegates (CPU oder GPU). Hierbei kommen „einfache“ Variablen zum Einsatz, die mit get/set-Methoden gelesen/beschrieben werden können. Diese Variablen sind privat im MainViewModel initialisiert und sind mit einem Unterstrich vor dem Namen markiert:

```

1 private var _threshold: Float = ObjectDetectorHelper.THRESHOLD_DEFAULT
2 private var _delegate: Int = ObjectDetectorHelper.DELEGATE_CPU
3 private var _maxResults: Int = ObjectDetectorHelper.MAX_RESULTS_DEFAULT
4 private var _model: Int = ObjectDetectorHelper.MODEL_TANKS

```

Die Default-Werte sind:

1. _threshold = THRESHOLD_DEFAULT = 0.8 → 80%
2. _delegate = DELEGATE_CPU = 0 → CPU
3. _maxResults = MAX_RESULTS_DEFAULT = 3 → drei Objekte gleichzeitig erkennbar
4. _model = MODEL_TANKS = 1 → Brauanlage.tflite

Diese Variablen sind privat und nur im MainViewModel verwendbar, um diese von außen (in den Tabs) zu verwenden, stehen im MainViewModel Getter- und Setter-Funktionen zur Verfügung.

Setter-Funktionen Im folgenden Codeausschnitt sind die Setter-Funktionen dargestellt, um die Variablen im Settings-Tab zu ändern.

```

1 fun setThreshold(value: Float) { _threshold = value }
2 fun setDelegate(value: Int) { _delegate = value }
3 fun setMaxResults(value: Int) { _maxResults = value }
4 fun setModel(value: Int) { _model = value }

```

Getter-Funktionen Im folgenden Codeausschnitt sind die Getter-Funktionen dargestellt, um die Variablen außerhalb des MainViewModels (im Kamera-Tab) zu lesen.

```

1 val currentThreshold: Float get() = _threshold
2 val currentDelegate: Int get() = _delegate
3 val currentMaxResults: Int get() = _maxResults
4 val currentModel: Int get() = _model

```

Der Aufruf der eingestellten Threshold im Kamera-Tab zum Beispiel lautet dann:

```

1 threshold = viewModel.currentThreshold

```

In Tabelle 1 ist eine Übersicht der wichtigsten privaten Variablen im MainViewModel gegeben.

Name	Datentyp	Beispiel
_tankTemperatures	Map<String, Float>	{„tank_1“=65.2, „tank_2“=33.4, „tank_3“=44.8}
_allValues	Map<String, Float>	{„tank_1“=65.2, ... „ventil_1“=1, ...}
_mqttMessages	List<String>	[„{„tank_1“:65.2“, ... , „ventil_1“:1}“, ...“]
_delegate	Int	0 = CPU; 1 = GPU
_threshold	Float	0.8 $\hat{=}$ 80 %
_maxResults	Int	maximal 3
_model	Int	0 = Brauanlage; 1 = EfficientDetLite0

Tabelle 1: Übersicht der privaten Variablen im MainViewModel

In Abbildung 2 ist der komplette Datenfluss vom Raspberry Pi als Broker bis zu den Tabs vereinfacht dargestellt.

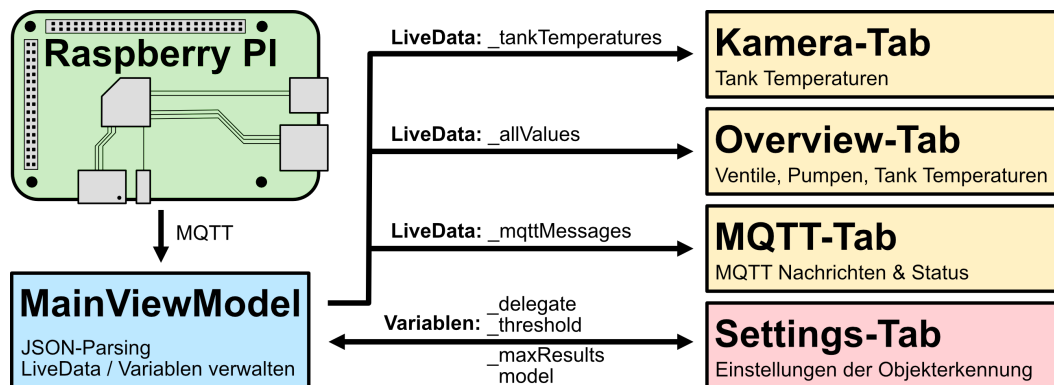


Abbildung 2: Überblick des Datenflusses der App
[1]

4.2 Navigation

Die MainActivity enthält außerdem ein NavHostFragment, das die einzelnen Tabs (Fragmente) rendert. Die Tab-Leiste ist als Top Navigation umgesetzt und wird per `setupWithNavController(...)` direkt mit dem NavController verbunden. Dadurch übernimmt die Navigation-Komponente das Umschalten zwischen Tabs automatisch (inklusive korrekter Titel-/Backstack-Verwaltung). Ein erneutes Antippen des bereits aktiven Tabs wird bewusst ignoriert, um unnötige Re-Loads zu vermeiden. Der Zurück-Button (`onBackPressed`) beendet die App.

4.3 Die verschiedenen Tabs

Die App soll mehrere verschiedenen Funktionen bieten, um diese übersichtlich darzustellen gibt es oben im Bildschirm eine Menu-Bar mit den passenden Symbolen zu jedem Tab. Mit diesem Menu kann zwischen den Tabs gewechselt werden. Die Übersicht dazu ist in Abbildung 3 dargestellt.

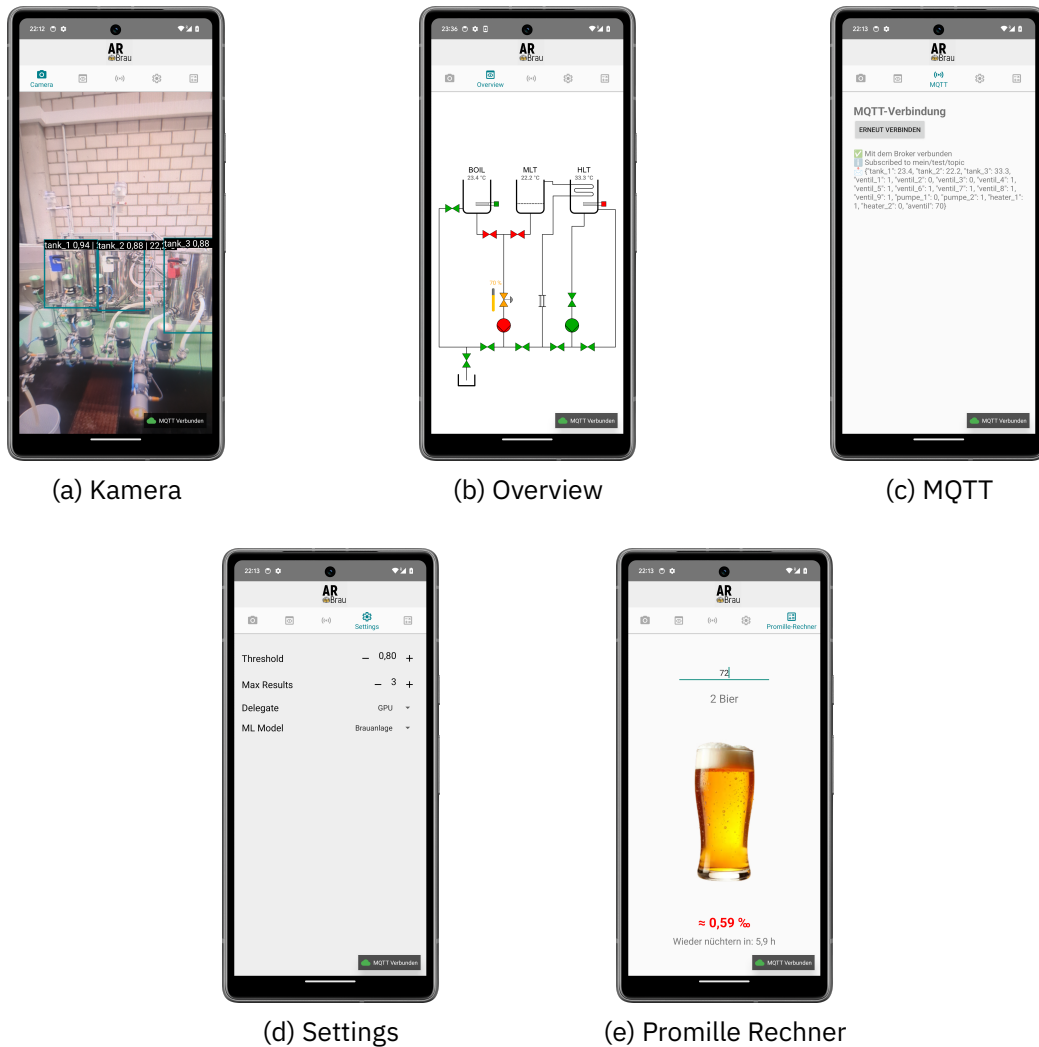


Abbildung 3: Die verschiedenen Tabs der Android App [2]

4.3.1 Kamera-Tab

Der Kamera-Tab wird standardgemäß als erstes beim Starten der App geöffnet und nutzt die CameraX-API, um den Kamera-Livestream des Smartphones anzuzeigen. Jedes Frame (einzelnes Bild) wird an die `ObjectDetectorHelper`-Klasse geschickt, welche das benutzerdefinierte Tensorflow-Lite Modell `Brauanlage.tflite` nutzt, um Tanks der Brauanlage zu erkennen und die Koordinaten der zugehörigen Bounding-Boxen zu berechnen. In der `OverlayView`-Klasse wird der Tank-Name, Erkennungssicherheit des Modells (Score) und die aktuelle Temperatur als Overlay auf einem erkannten Tank angezeigt. Die Temperaturdaten kommen regelmäßig per MQTT direkt von den Temperatur-Sensoren der Brauanlage im JSON-Format, zum Beispiel als: `{ "tank_1": 65.5, "tank_2": 70.2, "tank_3": 23.4, ... }` Diese Daten werden im `MainViewModel` verarbeitet. In der `CameraFragment`-Klasse werden die Temperatur-Daten aus dem `MainViewModel` kontinuierlich beobachtet.

4.3.2 Overview-Tab

Die Symbole der Sensoren und Aktoren ändern ihre Farbe basierend auf dem Zustand, der per MQTT übermittelten Schaltzustände. Initialisiert wird der Status in der App mit dem float-Wert 3f. Es gilt folgende Farbzuzuweisung, die auch in Code 1 zu sehen ist.

- **0**: aus/geschlossen → rotes Icon
- **1**: an/geöffnet → grünes Icon
- **sonst**: Startwert/unbekannt → graues Icon

```

1  val ventil1State = values["ventil_1"] ?: 3f
2      when (ventil1State){
3          0f -> binding.ventil1.setImageResource(R.drawable.vlv1_red)
4          1f -> binding.ventil1.setImageResource(R.drawable.vlv1_green)
5          else -> binding.ventil1.setImageResource(R.drawable.vlv1_gray)
6      }

```

Code 1: Zustand von ventil1 setzen

Für diesen Tab wurde die Brauanlage in einer 2D-Ansicht erstellt. In Abbildung 4 ist diese und die verwendete Namensgebung der Aktoren/Sensoren der MQTT-Nachricht in rotem Text abgebildet.

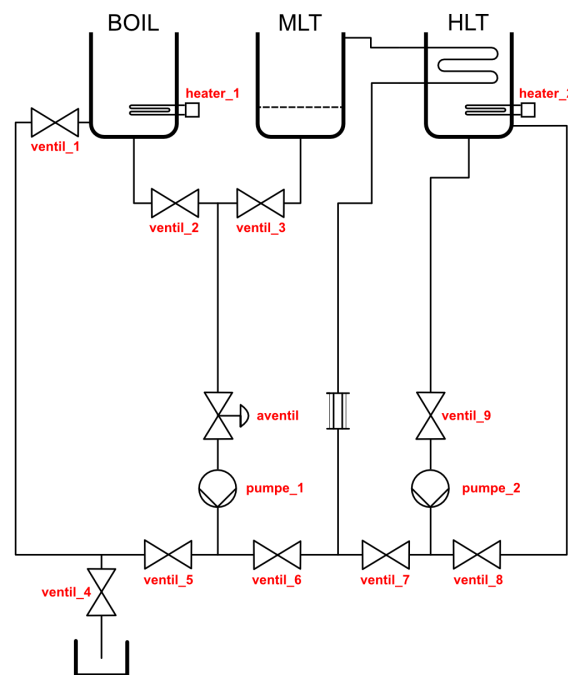


Abbildung 4: Namensgebung der Aktoren/Sensoren
[3]

4.3.3 MQTT-Tab

Dieser Tab dient dazu, die MQTT-Verbindung zu überprüfen und gegebenenfalls die Verbindung zum Broker neu aufzubauen. Dafür gibt es einen „Verbinden“-Button, der die Funktion `ViewModel.startMqtt()` aufruft. Das `MainViewModel` stellt die rohen MQTT-Daten im JSON-Format zur Verfügung.

4.3.4 Settings-Tab

Im Settings-Tab kann das Objekterkennungsmodell des Camera-Tabs noch detaillierter eingestellt werden. Es können die ML-Paramter wie `Treshold`, `MaxResults`, `Delegate` und `Model` manuell angepasst werden. Diese werden direkt ins `MainViewModel` geschrieben.

1. **Threshold:** Die minimal Erkennungssicherheit des Modells, um ein Objekt zu klassifizieren. Default: 0,8 bzw. 80 %
2. **MaxResults:** Anzahl der gleichzeitig erkannten Objekte. Default: 3
3. **Delegate:** Auswahl, ob die Objekterkennung auf der CPU oder GPU läuft. Die GPU bietet eine höhere Verarbeitungsgeschwindigkeit, jedoch ist diese bei alten Smartphones nicht vorhanden. Default: CPU
4. **Modell:** Auswahl des Objekterkennungsmodells, es gibt das Standardmodell `EfficientDetLite2` und das benutzerdefinierte `Brauanlagen-Modell`. Default: `Brauanlage`

4.3.5 Promillerechner-Tab

Durch Arbeiten an der Brauanlage ist es nicht auszuschließen, dass auch das ein oder andere Bier getrunken wird. Um im Anschluss die Verkehrstüchtigkeit zu gewährleisten, ist in der App ein Promillerechner integriert. Hierbei kann durch Klicken auf den „Bier-Button“ die Anzahl der getrunkenen Biere ausgewählt werden. Anschließend wird basierend auf dem Körpergewicht die Zeit berechnet, um wieder nüchtern zu werden. Die Anzahl der Biere kann durch einen langen Klick wieder auf 0 zurückgesetzt werden.

Wie in Gleichung 1 gezeigt, ergibt sich die maximale Zeit, um wieder nüchtern zu sein aus der Blutalkoholkonzentration (BAC) und der Abbaurate des menschlichen Körpers.

$$\begin{aligned} \text{BAC} &= \frac{12n}{0,68m}, \\ t &= \max\left\{0, \frac{\text{BAC}}{0,1}\right\} = \max\left\{0, \frac{12n}{0,68m \cdot 0,1}\right\}. \end{aligned} \quad (1)$$

Mit folgenden Zusammenhängen:

BAC Blutalkoholkonzentration in Promille (‰)

n Anzahl der getrunkenen Biere, es wird pro Bier ≈ 12 g reiner Alkohol angenommen

m Körpergewicht in Kilogramm

t maximale Zeit bis der Alkohol vollständig abgebaut ist

Hinweis: Es handelt sich um eine vereinfachte Abschätzung (kein medizinisches Gutachten). Individuelle Faktoren (z. B. Trinkverlauf, Geschlecht, Magenfüllung, tatsächlicher Alkoholgehalt jedes Bieres) bleiben unberücksichtigt.

4.4 MQTT Verbindungsstatus Overlay

Dieses Overlay ist in der rechten unten Ecke jederzeit in der App sichtbar. Es ist in der `MainActivity` initialisiert und beobachtet `viewModel.mqttConnected`. Wenn eine aktive MQTT Verbindung besteht, dann zeigt es „MQTT verbunden“ mit einem grünem Symbol und wenn keine Verbindung besteht, dann „MQTT Getrennt“ mit rotem Symbol.



(a) MQTT Verbinden Overlay



(b) MQTT Getrennt Overlay

5 Objekterkennungsmodell

Für die Objekterkennung kommt ein TensorFlow Lite Modell zum Einsatz, da dieses speziell für mobile Geräte optimiert ist. Es ist schnell, ressourcenschonend und läuft ohne Internetverbindung direkt auf dem Smartphone. Als Basis dient das Modell `EfficientDetLite0`, da es ein gutes Gleichgewicht zwischen Genauigkeit und Geschwindigkeit bietet. Es ist mit einem COCO-Datensatz vortrainiert, das bereits hundertausende Bilder mit verschiedenen Objekte aus dem Alltag enthält. Dadurch ist eine präzise Objekterkennung bereits gegeben, die nur noch auf die Tanks der Brauanlage „umtrainiert“ werden muss. Der große Vorteil hierbei ist, dass keine riesig Menge an Bildern benötigt wird und eine vergleichsweise kleine Sammlung an annotierten Bildern ausreicht, um ein präzises Modell zu erstellen.

Das benutzerdefinierte Modell verarbeitet Farbbilder mit einer Größe von 256 x 256 Pixel. Diese Bildgröße ist klein genug, um die Berechnungen effizient zu halten und gleichzeitig groß genug, um die Tanks zuverlässig zu erkennen. Als Zahlenformat wird `float32` verwendet. Dieses ist zwar rechenintensiver als die leichteren `float16` oder `int8` Formate aber bietet eine höhere Genauigkeit. Das ist vor allem notwendig, da der verwendete Trainingsdatensatz sehr klein ist. Die durchschnittliche Latenz[4] auf einem Google Pixel 6 beträgt:

- CPU: 61,30ms
- GPU: 27,83ms

5.1 Vorüberlegungen

Die drei Tanks sehen sich sehr ähnlich und sind daher schwer zu unterscheiden. Um die Unterscheidung zu vereinfachen und eindeutiger zu machen, werden farbige Griffhalter genutzt, die am Griff jedes Tanks angebracht werden können. Dadurch wird auch die Genauigkeit des späteren Objekterkennungsmodells stark profitieren. Bei der Umsetzung dieser Idee hat der Doktorand Simon Holzhauer geholfen, indem er die Markierungen entworfen und mit dem 3D-Drucker ausgedruckt hat. Die Zuweisung ist wie folgt:

- tank_1 (Boil) → blau
- tank_2 (MLT) → weiß
- tank_3 (HLT) → rot



Abbildung 6: Farbige Griffhalter aus dem 3D-Drucker
[5]

5.2 Datenset erstellen

Für die Erstellung eines hochwertigen Datensatzes zählt nicht nur die Anzahl der Bilder, sondern vor allem deren Qualität und Vielfalt. Damit das Objekterkennungsmodell möglichst genaue Ergebnisse liefert, sollten die Aufnahmen möglichst viele Variationen in den folgenden Punkten abdecken:

1. **Beleuchtung:** z. B. Tageslicht, künstliche Beleuchtung, Schatten
2. **Perspektive/Blickwinkel:** damit das Modell nicht nur von einer fixen Position funktioniert
3. **Hintergrund:** unterschiedliche Hintergründe sorgen dafür, dass sich das Modell nicht ausschließlich am Kontext orientiert
4. **Schärfe/Bildqualität:** jedoch sollten unscharfe oder stark verrauschte Bilder vermieden werden
5. **Einheitliche Skalierung:** damit das Objekt mit einer vergleichbaren Größenordnung arbeiten kann

Auf Grundlage dieser Überlegungen besteht der Datensatz aus insgesamt 104 Bildern der Brauanlage. Diese werden im Online-Tool RoboFlow hochgeladen. Über jeden Tank wird ein Rechteck (Bounding Box) gezeichnet und mit dem entsprechenden Label (tank_1, tank_2 und tank_3) versehen. Dieser Schritt ist in Abbildung 7 dargestellt ist. Davon werden 89 Bilder für das Training und 15 Bilder für die Validierung verwendet. Da diese Datenmenge sehr klein ist, wird der Datensatz in RoboFlow künstlich erweitert. Dabei kommen verschiedene Filter wie Zuschneiden (Crop), Farb- und Helligkeitsanpassung (Saturation, Brightness), Unschärfe (Blur) und Rauschen (Noise) zum Einsatz. Es gibt noch weitere Filter zur Auswahl, wie z.B. die Bilder um 90° zu drehen, horizontal/vertikal spiegeln oder Graufilter anzuwenden. Diese Funktionen wurden in einer vorherigen Version des Modells verwendet, jedoch führen diese zu vermehrten Fehlklassifizierungen. Vorallem der Graufilter hat das Modell mit den extra angefertigten Farbi- gen Griffhalter irritiert. Im der neusten Version, sind nur die oben genannten Filter verwendet.

Alle Bilder werden auf eine einheitliche Größe von 640 × 640 Pixel skaliert. Nach der Erweiterung umfasst der Datensatz 267 Trainingsbilder (95 %) und 15 Validierungsbilder (5 %). Der Export erfolgt im Pascal VOC-Format; dieser Ordner muss anschließend noch umstrukturiert werden.

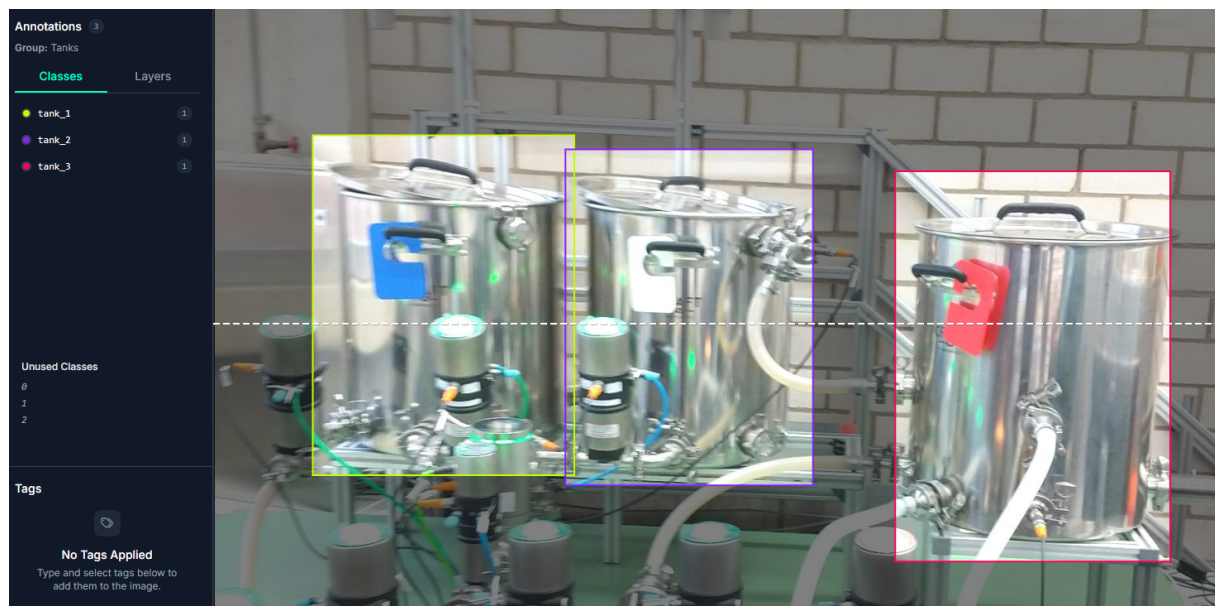


Abbildung 7: Bounding Boxen zeichnen mit RoboFlow
[6]

5.3 Mediapipe Model Maker

Für das Training des benutzerdefinierten Objekterkennungsmodells wird der von Google bereitgestellte Mediapipe Model Maker genutzt, um das EfficientDetLite0 Modell umzutrainieren. Dieses Training wird mit dem cloudbasierten Google Colab Notebook durchgeführt, da hier eine kostenlose online-GPU zur Verfügung steht. Dadurch läuft das Training deutlich schneller als mit der CPU vom PC/Laptop. Am Anfang wird geprüft, ob Python vorhanden ist (hier Version 3.12.11) und die nötigen Pakete werden geupdatet oder installiert.

```
1 !python --version
2 !pip install --upgrade pip
3 !pip install mediapipe-model-maker
```

```
1 from google.colab import files
2 import os
3 import json
4 import tensorflow as tf
5 assert tf.__version__.startswith('2')
6
7 from mediapipe_model_maker import object_detector
```

Anschließend muss der Datensatz als ZIP-Ordner auf Colab hochgeladen werden. In diesem Beispiel liegt ein Datensatz im Pascal VOC-Format vor und heißt `main_dataset.zip`. Hierbei ist es auch wichtig auf die richtige Ordnerstruktur zu achten, damit der Model Maker die Bilder und Labels korrekt erkennt. Gegebenfalls müssen neue Ordner angelegt und die Daten verschoben werden. Die Struktur ist wie folgt:

```
1 main_dataset/
2   train/
3     images/
4       image1.jpg
5       image2.jpg
6       [...]
7     Annotations/
8       image1.xml
9       image2.xml
10      [...]
11   valid/
12     images/
13       image3.jpg
14       image4.jpg
15       [...]
16     Annotations/
17       image3.xml
18       image4.xml
19      [...]
```

Wenn der Datensatz hochgeladen ist, kann dieser entpackt werden und die Pfade zu den Daten definiert werden.

```
1 !unzip main_dataset.zip
2
3 train_dataset_path = "main_dataset/train"
4 validation_dataset_path = "main_dataset/valid"
```

```
1 train_data = object_detector.Dataset.from_pascal_voc_folder(
2     'main_dataset/train',
3     cache_dir="/tmp/od_data/train",
4 )
5
6 validation_data = object_detector.Dataset.from_pascal_voc_folder(
7     'main_dataset/valid',
8     cache_dir="/tmp/od_data/validation"
9 )
```


Im nächsten Schritt kann das eigentliche Training durchgeführt werden. Es wird das vortrainierte MobileNetV2-Modell als Basis genutzt, da dieses für mobile Anwendungen optimiert ist. Die Trainingsparameter werden wie folgt festgelegt:

- **Batch Size** = 8: Anzahl der Bilder, die gleichzeitig verarbeitet werden
- **Learning Rate** = 0.3: Bestimmt die Lerngeschwindigkeit. Dieser Wert ist verhältnismäßig hoch, da das vorhandene Modell nur angepasst wird
- **Epochs** = 50: Wie oft der komplette Datensatz wiederholt wird
- **Export Directory** = 'exported_model': Speicherort für das trainierte Modell

```

1 hparams = object_detector.HParams(
2     batch_size=8,
3     learning_rate=0.3,
4     epochs=50,
5     export_dir='exported_model'
6 )
7
8 options = object_detector.ObjectDetectorOptions(
9     supported_model=object_detector.SupportedModels.MOBILENET_V2,
10    hparams=hparams
11 )
12
13 model = object_detector.ObjectDetector.create(
14     train_data=train_data,
15     validation_data=validation_data,
16     options=options
17 )

```

Ein Ausschnitt der Konsolenausgabe während des Trainings ist in Abbildung 8 dargestellt. Während dem Training lassen sich durch den Log schon die ersten Rückschlüsse ziehen: Man sieht, dass `total_loss` im Verlauf stark sinkt. Das bedeutet, dass das Modell lernt und sich auf die Trainingsdaten anpasst. Auch `cls_loss` und `box_loss` gehen runter. Das deutet daraufhin, dass die Klassifikation und das Zeichnen der Bounding Boxen der Tanks immer besser funktioniert. Die Validierungsverluste (`val_total_loss`) bleiben jedoch relativ hoch. Das bedeutet, dass das Modell mit unbekannten Bildern schlechter lernt als mit den genutzten Trainingsbildern. Das ist hauptsächlich auf die geringe Bilder-Anzahl zurückzuführen.

```

.....] - 63s 2s/step - total_loss: 0.3136 - cls_loss: 0.2232 - box_loss: 7.0977e-04 - model_loss: 0.2587 - val_total_loss: 1.5699 - val_cls_loss: 1.4154 - val_box_loss: 0.0019 - val_model_loss: 1.5289
.....] - 63s 2s/step - total_loss: 0.2800 - cls_loss: 0.1953 - box_loss: 5.9551e-04 - model_loss: 0.2251 - val_total_loss: 1.5838 - val_cls_loss: 1.4330 - val_box_loss: 0.0019 - val_model_loss: 1.5289
.....] - 63s 2s/step - total_loss: 0.2629 - cls_loss: 0.1804 - box_loss: 5.5273e-04 - model_loss: 0.2080 - val_total_loss: 1.6078 - val_cls_loss: 1.4604 - val_box_loss: 0.0018 - val_model_loss: 1.5529
.....] - 63s 2s/step - total_loss: 0.2484 - cls_loss: 0.1672 - box_loss: 5.2597e-04 - model_loss: 0.1935 - val_total_loss: 1.6272 - val_cls_loss: 1.4805 - val_box_loss: 0.0018 - val_model_loss: 1.5723
.....] - 62s 2s/step - total_loss: 0.2570 - cls_loss: 0.1703 - box_loss: 6.3559e-04 - model_loss: 0.2021 - val_total_loss: 1.6901 - val_cls_loss: 1.5427 - val_box_loss: 0.0018 - val_model_loss: 1.6352
.....] - 63s 2s/step - total_loss: 0.2300 - cls_loss: 0.1497 - box_loss: 5.0732e-04 - model_loss: 0.1751 - val_total_loss: 1.7093 - val_cls_loss: 1.5667 - val_box_loss: 0.0018 - val_model_loss: 1.6544
.....] - 62s 2s/step - total_loss: 0.2090 - cls_loss: 0.1339 - box_loss: 4.0349e-04 - model_loss: 0.1541 - val_total_loss: 1.7309 - val_cls_loss: 1.5835 - val_box_loss: 0.0019 - val_model_loss: 1.6760
.....] - 63s 2s/step - total_loss: 0.2140 - cls_loss: 0.1366 - box_loss: 4.5004e-04 - model_loss: 0.1591 - val_total_loss: 1.7181 - val_cls_loss: 1.5794 - val_box_loss: 0.0017 - val_model_loss: 1.6632
.....] - 63s 2s/step - total_loss: 0.1951 - cls_loss: 0.1219 - box_loss: 3.6705e-04 - model_loss: 0.1402 - val_total_loss: 1.7504 - val_cls_loss: 1.6118 - val_box_loss: 0.0017 - val_model_loss: 1.6956
.....] - 63s 2s/step - total_loss: 0.1878 - cls_loss: 0.1153 - box_loss: 3.5152e-04 - model_loss: 0.1329 - val_total_loss: 1.7691 - val_cls_loss: 1.6255 - val_box_loss: 0.0018 - val_model_loss: 1.7143
.....] - 63s 2s/step - total_loss: 0.1837 - cls_loss: 0.1112 - box_loss: 3.5205e-04 - model_loss: 0.1288 - val_total_loss: 1.7970 - val_cls_loss: 1.6545 - val_box_loss: 0.0018 - val_model_loss: 1.7421
.....] - 62s 2s/step - total_loss: 0.1744 - cls_loss: 0.1046 - box_loss: 2.9915e-04 - model_loss: 0.1196 - val_total_loss: 1.8363 - val_cls_loss: 1.6948 - val_box_loss: 0.0017 - val_model_loss: 1.7814
.....] - 62s 2s/step - total_loss: 0.1700 - cls_loss: 0.1004 - box_loss: 2.9673e-04 - model_loss: 0.1152 - val_total_loss: 1.8671 - val_cls_loss: 1.7209 - val_box_loss: 0.0018 - val_model_loss: 1.8123
.....] - 63s 2s/step - total_loss: 0.1756 - cls_loss: 0.1037 - box_loss: 3.4053e-04 - model_loss: 0.1208 - val_total_loss: 1.9128 - val_cls_loss: 1.7686 - val_box_loss: 0.0018 - val_model_loss: 1.8579
.....] - 63s 2s/step - total_loss: 0.1724 - cls_loss: 0.1007 - box_loss: 3.3702e-04 - model_loss: 0.1176 - val_total_loss: 1.9624 - val_cls_loss: 1.8230 - val_box_loss: 0.0017 - val_model_loss: 1.9076
.....] - 62s 2s/step - total_loss: 0.1665 - cls_loss: 0.0954 - box_loss: 3.2446e-04 - model_loss: 0.1117 - val_total_loss: 2.0142 - val_cls_loss: 1.8717 - val_box_loss: 0.0018 - val_model_loss: 1.9594
.....] - 63s 2s/step - total_loss: 0.1573 - cls_loss: 0.0892 - box_loss: 2.6519e-04 - model_loss: 0.1024 - val_total_loss: 2.0276 - val_cls_loss: 1.8854 - val_box_loss: 0.0017 - val_model_loss: 1.9728
.....] - 63s 2s/step - total_loss: 0.1658 - cls_loss: 0.0945 - box_loss: 3.2939e-04 - model_loss: 0.1110 - val_total_loss: 2.0859 - val_cls_loss: 1.8654 - val_box_loss: 0.0017 - val_model_loss: 1.9511
.....] - 62s 2s/step - total_loss: 0.1556 - cls_loss: 0.0869 - box_loss: 2.7914e-04 - model_loss: 0.1088 - val_total_loss: 1.9703 - val_cls_loss: 1.8292 - val_box_loss: 0.0017 - val_model_loss: 1.9155
.....] - ETA: 48s - total_loss: 0.1601 - cls_loss: 0.0915 - box_loss: 2.7672e-04 - model_loss: 0.1053

```

Abbildung 8: Konsolenausgabe während des Trainings

[7]

Abschließend kann das Model exportiert und heruntergeladen werden. Dabei wird das Modell in das .tflite-Format konvertiert, welches für die Android App benötigt wird. Der Export erfolgt mit dem folgenden Code:

```
1 model.export_model()  
2 !ls exported_model  
3 files.download('exported_model/model.tflite')
```

6 Schluss

Abschließend lässt sich das Projekt zusammenfassen: Die Kommunikation der Prozesswerte erfolgt zuerst mit OPC UA von der SPS zum Raspberry Pi. Anschließend werden über den Acces Point (FritzBox) die Prozesswerte über den MQTT-Broker (Raspberry Pi) für Geräte im Umfeld des WLAN-Netztes bereitgestellt. Ein Smartphone, das die Android App installiert hat, muss mit dem WLAN-Netzwerk des Acces Points verbunden sein und kann dadurch die Prozesswerte empfangen. Diese Werte werden in der App verarbeitet und aufbereitet, sodass sie in der App in verschiedenen Tabs angezeigt werden können. Außerdem verfügt die App über eine benutzerdefinierte Objekterkennung, welche die drei Tanks der Brauanlage erkennt. In der Kamera-Ansicht werden in Echtzeit die Tanks mit Bounding-Boxen mit Labels entsprechend gekennzeichnet. Zusätzlich werden die aktuellen Temperaturwerte als Overlay auf den Tanks angezeigt.

Trotz anfänglicher Schwierigkeiten vorallem beim Objekterkennungsmodell, erfüllt die App fast alle Kriterien. Es fehlt lediglich die Füllstandsanzeige und der Kobold, der den Topf rührt. Die Radar-Sensoren, die für die Füllstandsmessung vorgesehen sind zum aktuellen Stand defekt und können nicht integriert werden. Die Koboide sind nicht implementiert, da in der App keine Rezeptdaten vorliegen. Deshalb kann nicht bestimmt werden, wann die Tanks gerührt werden müssen, was durch die Koboide visualisiert werden soll.

Die ersten Objekterkennungsmodelle wurden mit dem von Google bereitgestellten TensorFlow Lite Modelmakers trainiert. Jedoch hat das Training wegen Bugs seitens Google nicht funktioniert. Wir haben viele Workarounds probiert, wie ältere Versionen des Modelmakers, YOLOV5 / YOLOV8 Modelle und verschiedene Entwicklungsumgebungen (cleanes virtual environment, Docker oder Google Codelab). Schließlich war der MediaPipe Model Maker die Lösung, hierbei hat die offizielle Version nicht auf Anhieb funktioniert. Mit ein paar Anpassungen im Code konnte schließlich das aktuelle Brauanlage.tflite Modell trainiert werden.

7 Ausblick

Mit diesem Projekt ist eine erste Basis entstanden, auf der man in Zukunft gut weiterarbeiten kann. Das Objekterkennungsmodell kann zum Beispiel noch präziser und sicherer werden, wenn deutlich mehr Bilder für das Training genutzt werden. Auch weitere Messwerte wie die Füllstände der Behälter können integriert werden, allerdings sind die dafür vorgesehenen Radar-Sensoren aktuell defekt und konnten deshalb nicht eingebunden werden. Ebenso wäre es sinnvoll, die App mit Rezeptdaten zu verknüpfen, damit die Arbeitsschritte beim Brauen in der App vorliegen und angezeigt werden können. Hiermit kann auch der geplante „Rühr-Kobold“ implementiert werden. Denkbar wäre außerdem, dass die App nicht nur Zustände anzeigt, sondern auch aktiv in die Anlage eingreift, etwa durch das Schalten von Ventilen oder Pumpen. Diese Steuerung ist im Moment aus Sicherheitsgründen noch nicht umgesetzt.

Literatur- und Abbildungsverzeichnis

- [1] Niklas Felhauer. *Datenfluss des MainViewModels*. Eigene Darstellung, vereinfacht dargestellt. 2025.
- [2] Niklas Felhauer. *Die verschiedenen Tabs der Android App*. Eigene Darstellung. Screenshot aus der Android-App. 2025.
- [3] Niklas Felhauer. *Nummerierung der Aktoren/Sensoren in der Android App*. Eigene Darstellung. Screenshot aus der Android-App. 2025.
- [4] Google. *Google MediaPipe Tutorial*. Zugriff am 18. Juni 2025. 2025. URL: https://ai.google.dev/edge/mediapipe/solutions/vision/object_detector/index?hl=de#models.
- [5] Niklas Felhauer. *Farbige Griffhalter aus dem 3D-Drucker*. Eigene Darstellung. Eigene Aufnahme. 2025.
- [6] Niklas Felhauer. *Bounding Boxen zeichnen mit Roboflow*. Eigene Darstellung. Screenshot aus der Roboflow. 2025.
- [7] Niklas Felhauer. *Konsolenausgabe während des Trainings*. Eigene Darstellung. Screenshot aus der Konsolenausgabe. 2025.